



Ballistics and Scoring

Programmer's Reference Guide

Version 1.17.0 (build 26) • August 2026

developed by Dr Artur Krukowski

Contents

1. Architecture overview

BAS is a single-module Kotlin Android app (package `com.rfsat.bas`, applicationId `com.BAS`, minSdk 26, compileSdk 36, AGP 8.9.1, Kotlin 2.1.0, CameraX 1.4.2, Material 1.12.0, view binding, no DI framework). It is the union of two applications: the scoring half derives from STS, the ballistics half from VTB, merged with STS as the base because its profile classes were already written as supersets of VTB's.

Five screens share a `BottomNavigationView` through a common `BaseActivity`; analysis runs in coroutines on `Dispatchers.Default`; persistence is `SharedPreferences` with `Gson` for structured payloads. No server component — external I/O is Bluetooth LE (Kestrel, rangefinders), HTTP over a camera's own Wi-Fi (GoPro, action cameras), RTSP/UDP video, and an optional user-keyed AI scoring endpoint.

Package / class	Responsibility
<code>BasApp</code>	Process entry: crash handler first, safe-mode gate, guarded init of <code>Logger</code> / <code>Theme</code> / <code>Units</code> / <code>Range</code> / <code>Speaker</code> , and restore of <code>ScoringSession</code> , <code>AnalysisSession</code> and <code>EnvironmentManager</code> .
<code>ui.BaseActivity</code>	<code>Theme</code> , full-screen, cutout-inset padding, bottom-nav wiring, swipe-to-switch, keep-awake, remote-trigger key handling, themed <code>notifyUser()</code> .
<code>ui.MainActivity</code>	Home: crash-report dialog, active-setup summary, <code>Range</code> mode entry.
<code>ui.RangeActivity</code>	Full-screen glance: wind and group corrections (clicks + MRAD/MOA) and score, polled live.
<code>capture.CaptureActivity</code>	Ballistics screen: camera lifecycle, auto-trigger, import, source routing, stream capture, auto-collect, analysis orchestration.
<code>capture.TrailExtractor</code> / <code>FastFrameDecoder</code>	Reference averaging and one-pass <code>MediaCodec</code> decode to downsampled luminance.
<code>wind.WindEstimator</code> / <code>TracerWindEstimator</code>	Vapour drift-rate fit; tracer/projectile lag-rule WLS.
<code>ballistics.BallisticsEngine</code>	Point-mass G1 integration, zero solving, TOF, drag-decayed distance mapping.
<code>detect.*</code>	Scoring half: registration, homography, ring fitting, hole detection, lens distortion, RTSP client.
<code>scoring.*</code>	<code>ScoringEngine</code> , <code>ScoringSession</code> , <code>GroupStatistics</code> , <code>CorrectionCalculator</code> , <code>ShotDistribution</code> .
<code>targets.*</code> / <code>rules.*</code>	Target faces and rule sets, their repositories and editors.
<code>results.*</code>	Both results screens: scoring <code>ResultsActivity</code> and <code>BallisticsResultsActivity</code> , <code>AnalysisSession</code> , <code>AdjustmentCalculator</code> .
<code>environment.*</code>	<code>EnvironmentManager</code> , <code>KestrelProvider</code> , <code>RangefinderModel</code> / <code>Link</code> / <code>Probe</code> / <code>Ui</code> , <code>DistanceConfig</code> .
<code>profiles.*</code>	Unified Rifle/Bullet/Scope data classes, <code>ProfileRepository</code> , catalogues, <code>ProfileActivity</code> .
<code>cloud.*</code>	Optional AI second opinion / outright scoring, user-keyed.
<code>backup.AppBackup</code>	Whole-app backup and restore across every <code>SharedPreferences</code> store.
<code>log.*</code>	File-backed <code>Logger</code> and the <code>Log</code> screen.

2. Build, versioning and delivery

- Gradle Kotlin DSL; CI (`.github/workflows/android-ci.yml`) is release-only: static checks, then unit tests against the `RELEASE` variant, then `assembleRelease` + `bundleRelease`, uploading APK, AAB and the R8 mapping.
- Signing uses four repository secrets (`ANDROID_KEYSTORE_BASE64` / `_PASSWORD` / `_KEY_ALIAS` / `_KEY_PASSWORD`). A keystore is not bound to a package name, so the STS or VTB key signs BAS unchanged; only applicationId must be new.
- Versioning is `<brand>.<major>.<minor>`: brand 1 = BAS and does not change; major on a new feature (minor resets); minor on a correction. `versionCode` increments on every shipped artifact and never repeats.
- Each release ships as a full-state zip named `BAS_v<brand>_<major>_<minor>.zip`. The version line at the foot of `Settings` identifies the installed binary — trust it over filenames.
- `.gitattributes` normalises text to LF and pins `.sh/.py/gradlew` to LF on checkout; a shell script checked out CRLF fails on Linux with 'bad interpreter'.

3. Startup diagnostics

BasApp installs the uncaught-exception handler BEFORE anything else, so a failure in initialisation is still reported. A safe-mode gate skips stored-session restore when the previous launch crashed. Every init is wrapped in runCatching: shared startup code must never kill a screen it merely decorates.

Restored at startup: ScoringSession (shots and selection), AnalysisSession (last wind solution and history), EnvironmentManager (last Kestrel reading, with its age), RangeSettings, Units and Theme.

4. Capture pipeline (ballistics)

- Shot break (AudioShotDetector): decode audio to PCM, first 5 ms window $\geq 60\%$ of the global peak; falls back to the manual field.
- Decode (FastFrameDecoder): single sequential MediaCodec pass, $\sim 12\times$ faster than per-frame seeks; container rotation applied during conversion.
- Reference (TrailExtractor): average up to 8 pre-shot frames; single-frame lookup as fallback.
- The phone camera is rebound on EVERY resume, not only when videoCapture is null — returning from another screen leaves the use cases unbound while the field is still set, which is the black-preview fault fixed in 1.14.0.
- PreviewView runs in COMPATIBLE (TextureView) mode alongside the stream SurfaceView.

5. Analysis pipeline (wind and solution)

- Vapour (WindEstimator): after a settle period of $1.2\times\text{TOF}$, windowed weighted fits of drift rate; crosswind = drift rate $\times$ effective distance $\times$ windScale (clamped 0.2–5); samples ≥ 15 m/s rejected.
- Tracer / projectile (TracerWindEstimator): $z = W \cdot (t - x/v_0) + \theta_0 \cdot x$ by two-parameter WLS; the $\theta_0 \cdot x$ term absorbs boresight bias so it cannot masquerade as wind.
- If a point mode yields fewer than four observations the analysis re-runs in VAPOR and uses that if it tracks better.
- A declared camera red dot is excluded via centerExclusionRadiusPx; declared stabilisation raises a warning.
- Corrections (AdjustmentCalculator): averaged wind $\rightarrow$ trajectory $\rightarrow$ clicks, with the same correction exposed in MRAD/MOA for the glance screen.

6. Scoring pipeline

- Registration (TargetRegistration, Homography): four-corner or ring-family fit produces a projective map from target-plane millimetres to image pixels. Everything downstream works in millimetres.
- Ring fitting (RingFinder, RingFamilyFit, EllipseFit): concentric ring radii should form an arithmetic progression; the departure yields both the scale and the lens-distortion estimate (LensDistortion).
- Detection (HoleDetector, SourceHoleDetector, LocalBackground, PunctureCheck): difference against a clean reference where one exists, otherwise segmentation against local background. FocusedRemeasure sizes each hole at half its own height above background rather than an absolute level.
- Scoring (ScoringEngine): hole centre and gauge against the face's ring geometry under the rule set; decimal or integer; inner tens optional.
- Session (ScoringSession): shots, selection, persistence, undo, reindex. removeShots matches by IDENTITY — a reindex makes distinct shots compare equal.
- Statistics and correction (GroupStatistics, CorrectionCalculator, ShotDistribution): group centre and dispersion, the sight correction in clicks or as rear-sight movement, and the distribution that separates a sight error from a process error.

7. Target faces and rule sets

TargetFace carries geometry (ring radii, aiming mark, card size); RuleSet carries convention (gauge diameter, decimal scoring, shot counts, inner tens, penalties). Both ship as catalogues (TargetCatalog, RuleCatalog), both are user-editable through their repositories, and both are addressed by id so a session keeps the face and rules it was scored with.

8. Camera abstraction

Cameras are selected identically on both tabs and defaulted in Settings.

Type	Path
CameraType	PHONE, GOPRO, TACTACAM, SHOTKAM, RTSP with capability flags (hasHost, canDownload, canScan, isGoPro, canLive).
CameraConfig	Persists the selected type and per-camera host; maps a type to a CameraFileImporter preset.
CameraUi	Shared chooser and host prompt.
CameraWifi	Binds the process to an internet-less AP (TRANSPORT_WIFI without NET_CAPABILITY_INTERNET, CHANGE_NETWORK_STATE), with a longer wait when auto-reconnect is on.
CameraFileImporter	latestUrl / downloadUrl / downloadLatest over undocumented action-cam endpoints (Novatek cmd=3015, DCIM listings).
CameraScanner	Wide-net discovery: gateway detection, TCP sweep of candidate hosts and ports, then deep HTTP probe; logs every result.

Auto-collect polls latestUrl on the selected camera; a new identifier means recording stopped, so the clip is downloaded and analysed automatically. The identifier present at arming is ignored.

9. GoPro integration

GoPro (HERO9 and later) is the only camera here with a published interface, so it is implemented against the spec rather than probed.

```
GET /gopro/media/list      JSON media list
GET /videos/DCIM/<dir>/<file>  download
GET /gopro/camera/shutter/start|stop
GET /gopro/camera/digital_zoom?percent=N (fallback range_pcmt=N)
GET /gopro/camera/presets/load?id=N · /state · /keep_alive
GET /gopro/camera/stream/start then read udp://:8554 (MPEG-TS)
```

GoProPreviewStream demuxes the transport stream (PAT → PMT → H.264 PID, PES reassembly, SPS/PPS via the scoring half's SpsDimensions parser) and renders to a Surface with MediaCodec, with a keep-alive so the link does not sleep. The preview is 480p (HERO9) / 720p (HERO11): aiming, not scoring — full resolution exists only in the recorded file.

10. Rangefinder and distance

Distance has three routes, layered so coverage does not depend on decoding vendor protocols.

- KESTREL_BRIDGE — Leica, Vortex and SIG BDX-X push range into a Kestrel 5700 Elite, which BAS already speaks; one link covers three brands.
- Direct models — SIG_KILO, LEICA, VORTEX, TERRAPIN, FIRE4000, GENERIC. Device selection is by advertised/bonded name hints.
- MANUAL — typed, or taken from the rig's zero distance (editable and saved back to the rifle profile).

RangefinderLink subscribes to every notifying characteristic and decodes each frame by trying 16/32-bit values, little- and big-endian, against metre / decimetre / yard / tenth-yard scales, accepting only 5–4000 m. That heuristic is not sufficient alone — a 21.5 °C frame decodes as a plausible 215 m — so the FIRST accepted reading is offered for confirmation against the device display; on acceptance the characteristic UUID and scale are locked in

DistanceConfig and every later reading uses that pairing. RangefinderProbe is the diagnostic counterpart: it enumerates and logs everything without deciding anything.

11. Kestrel — Bluetooth protocol

KestrelProvider supports two meter families through one connect path: the 5700 Elite / 5700X with LiNK (and the Ruger co-brand), which bond normally, and the DROP D3, which is advertising-only and never appears in paired devices, so a short scan is used. It connects, discovers every service, reads every readable characteristic and logs UUIDs with raw hex; the DROP service base is 12630000-cc25-497d-9854-9b6c02c77054, and the Bluetooth SIG Environmental Sensing service (0x181A) is used where present. Readings persist with their age.

12. Persistence and profiles

The equipment store is unified and single. STS's RifleProfile, BulletProfile, ScopeProfile and ClickUnit were already field-supersets of VTB's, with identical Gson field names, so the ballistics code binds to them with no field changes — this is why STS became the merge base. ProfileSet groups the three; ProfileRepository owns migration, seeding and the active selection.

Seeded sets are applied on first launch, the first being the Rimfire trainer (Ruger Precision Rimfire .22LR / Federal Champion 40 gr / Vector Optics Continental 5-30x56), so there is no first-run picker and no empty state.

Stores: bas_prefs (shared chrome), bas_units, bas_theme, bas_range, bas_camera, bas_distance, plus the profile, target, rule, scoring and analysis stores.

13. Data-class evolution rules (Gson)

- Adding a field is safe: give it a default. Gson leaves it at the default for payloads written by older builds.
- Renaming a field is a data loss. Keep the stored key and rename only the accessor, or migrate explicitly in the repository.
- Deprecated fields stay for reading (zeroDistanceYards) and are migrated on load.
- Enums persist by NAME: order may change, names may not.

14. UI shell and navigation

- Five tabs at Material's cap: Home, Ballistics, Scoring, Results, Settings. Targets, rules, log, camera defaults, range options and backup are sections in Settings; Exit sits outside the capped menu.
- The Results tab is context-aware: from the ballistics flow it opens BallisticsResultsActivity, otherwise the scoring ResultsActivity; both carry a Scoring / Ballistics switch.
- ReticleDrawer is the single source for reticle rendering, shared by CrosshairView (scoring) and CrosshairOverlayView (ballistics), coloured from the basReticleColor theme attribute with a dark halo for contrast over paper and aiming mark alike.
- Content is padded by the displayCutout inset — the app draws into the cutout strip, so a top-centre selfie camera would otherwise cover controls.
- Settings sections fold under tagged headings; opening Profile sets also opens Firearm, Load and Sight.

15. Hands-free subsystem

Component	Role
RangeSettings	speak, keepAwake, autoReconnect, autoShowResults, autoCollect, remoteTrigger, skipConfirm.
Speaker	TextToSpeech wrapper; silent unless speak() is on. Engine declared in <queries> for Android 11+.
Corrections	Formats a SightCorrection or ScopeAdjustment as arrows+clicks, as MRAD/MOA, and as terse speech.
BaseActivity.onRemoteTrigg	Volume / camera / media / headset / enter keys fire the screen's primary action.

Component	Role
er	
RangeActivity	Polls the live session and speaks each new shot.

16. Static checks (tools/kotlin_checks.py)

A pre-compiler gate run first in CI. Each rule exists because the corresponding mistake reached a build and was reported somewhere other than its cause.

Gate	What it catches
1-3	Semantic checks; return@label naming a lambda that is not enclosing (parses backward over balanced parentheses); when-over-enum without else.
4	binding.<id> not present in that screen's layout, following <include>.
5-7	Import hygiene, cross-file private visibility, language level (no break/continue in inline lambdas on Kotlin 2.1).
8	An import placed after a top-level declaration — Kotlin requires imports directly after package.
9	A top-level object/class used unqualified from another package; tracks every declaring package so a same-package duplicate is not a false positive.
10	res/values/*.xml well-formedness and unescaped apostrophes or quotes in a <string> (aapt2 rejects them).

17. Chart distance mapping

Wind samples are mapped to downrange distance through a drag-decayed integral capped at the target: the bullet stops there, but the trail keeps drifting and still measures wind. The chart therefore plots against distance while the bullet is in flight and switches to time after the shot once it has arrived, because every post-impact sample shares the target distance and would otherwise stack on one vertical line.

18. Whole-app backup

AppBackup serialises every SharedPreferences store into one JSON payload with a format version, and refuses a payload written by a newer format. API keys are deliberately EXCLUDED from backup and from device transfer: they are encrypted against a keystore master key that does not travel, so a restored copy would be unreadable ciphertext rather than a working key.

19. Google Play build configuration

- applicationId com.BAS is permanent once published; the Kotlin/resource namespace stays com.rfsat.bas, so no source moves.
- R8 is enabled for release with NDK debugSymbolLevel = SYMBOL_TABLE; keep mapping.txt per build — an old release stays diagnosable after Play overwrites what it holds.
- tools/generate_icons.py emits the 512 px Play icon and the adaptive foreground and themed monochrome at every density from ONE drawing; drawing them separately is how a launcher icon and a store listing drift apart, which happened once and is why the generator exists.

20. Known pitfalls

- Theme attributes consumed by Material as resource references (textColorSecondary) must never be literals — inflation crashes before any code runs.
- MediaMetadataRetriever on content:// URIs can kill the process natively — copy to a cache file first.
- TextureView.getBitmap() off the thread owning the surface returns a BLANK bitmap rather than failing: the read-back must be posted to the main thread.
- A default Spinner selection at position 0 fires no callback; a swallow-first guard then eats the user's first real selection.

- requestNetwork for an internet-less AP is never satisfied unless NET_CAPABILITY_INTERNET is removed, and needs CHANGE_NETWORK_STATE.
- Renaming a class without its call sites compiles nowhere but reads fine in review — gate 9 exists for exactly this.
- A seeded profile applied silently as the default decides behaviour far downstream: an air-rifle rig makes the ballistic solution invalid and the wind chart is suppressed by design.
- Verify a fix by extracting the SHIPPED zip, not the working tree.

21. References

- Open GoPro HTTP specification (gopro.github.io/OpenGoPro).
- Bluetooth SIG Environmental Sensing Service 0x181A.
- ISSF and NRA rulebooks for the shipped rule sets — nominal published figures, editable in the app.